

The Reproducibility Test as Standalone Integrative Procedure

Specification: Formalizing the Operational Test That Verifies A1.07 × A1.10 Composition Through Replay-Verification in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the operational test for **reproducibility** — the integrative property that emerges when path retraceability and the determinism contract compose — as a standalone replay-verification procedure separable from the component tests for each commitment.

Abstract

Reproducibility is the property that, given a substrate state at t1 and the recorded sequence of changes from t1 to t2 with full provenance and recorded LLM outputs, the substrate state at t2 can be reproduced exactly through replay from the t1 state. It is the integrative verification of two architectural commitments — path retraceability (§5 of the source paper) and the determinism contract (§4.1, §11.3) — working together. Component tests for each commitment do not certify that the composition holds; the reproducibility test does. It exercises both commitments end-to-end and produces a binary outcome: replay reproduces the recorded state, or it does not. This note specifies the test as a standalone integrative procedure: what it verifies, the procedural steps, pass and fail criteria, the canonical anti-patterns it detects, how it integrates with deployment verification, and its limits. The note closes Phase A5 of the Series A derivation programme.

1. Why the reproducibility test needs to be formalized as standalone

Path retraceability is the substrate-side commitment that the causal path from any piece of content to its antecedents is reconstructable from substrate content alone (§5). The determinism contract is the substrate-side commitment that same state, same rules, and same recorded LLM outputs yield equivalent substrate writes, reads, and conflict outcomes (§4.1, §11.3). Each is independently testable and has its own sibling test in Phase A5.

Reproducibility is what these commitments produce when they compose. With path retraceability alone, the substrate carries provenance describing what happened, but offers no guarantee the described history can be reconstructed; the recorded path is a description, not a procedure. With the determinism

contract alone, substrate behavior is well-defined under same inputs, but inputs are not guaranteed recoverable. Composing the two yields the integrative property: a recorded history is sufficient to reconstruct the resulting state, because the recording is complete (retraceability) and the replay is well-defined (determinism).

Because reproducibility is the integrative consequence, it is not verified by the component tests taken individually. A deployment can pass path-retraceability, the determinism tests, provenance-completeness, and the four-accountability-questions test, and still fail reproducibility — most commonly when provenance is individually well-formed but jointly incomplete, or when cells are deterministic in isolation but exhibit emergent non-determinism at composition boundaries. Only the integrative test catches these cases.

Naming the test as standalone closes Phase A5 and gives implementers, auditors, and critics a single citable procedural specification. *Auditable AI*, *reproducible AI*, and *replayable AI workflows* are terms in active use across AI governance and engineering literatures; specifying the test as a defined procedure with defined inputs, replay logic, pass/fail criteria, and diagnostic resolution to specific anti-patterns is what makes it citable in those literatures.

2. The architectural commitment under test

The reproducibility test verifies the composition of two foundational commitments and the mediator role that binds them.

Path retraceability (§5). The substrate carries, for every piece of content, a provenance record sufficient to reconstruct the causal path back to its antecedents. The fields include the writer (a human, or a cell operating under a named orchestration rule), the timestamp, the rule reference where the writer was a cell, the antecedent substrate content drawn on, and — where the writer was a cell whose rule mediated an LLM output — the LLM output itself, recorded under the AI-as-substrate-mediator commitment.

The determinism contract (§4.1, §11.3). The substrate satisfies five guarantees: read determinism, cell-behavior determinism (modulo recorded LLM outputs), write addressability, conflict preservation, and substrate-as-source-of-truth. Cell behavior is deterministic at the substrate-write layer: given the same substrate state, rule, and recorded LLM output, the cell produces the same writes. Allowed non-determinism is bounded — LLM outputs are non-deterministic, but the mediator role's recording requirement makes the otherwise non-deterministic output an addressable input to deterministic cell logic.

The composition. Reproducibility holds when the two commitments hold together, mediated correctly. Given a substrate state at t1 and a recorded sequence of changes from t1 to t2 — with full provenance and every informing LLM output recorded as substrate-addressable content — the substrate state at t2 can be reproduced exactly through replay from X. Replay is well-defined because determinism holds; inputs are recoverable because retraceability holds; bounded LLM non-determinism is reproducible because the mediator role records it. The test verifies the composition by exercising it.

3. The test procedure

The test proceeds in seven steps, each verifiable from substrate alone.

(1) Capture substrate state X at t_1 — entities, relationships, decisions, conflict records, rationale, and orchestration rules — at the source-of-truth content scope.

(2) Allow normal operations from t_1 to t_2 : human exercises of inspect, modify, and override rights; AI-mediated cell processing under orchestration rules; trigger events; conflict-resolution actions. The interval is chosen to cover representative activity, not to engineer a favorable test case.

(3) Capture substrate state Y at t_2 with the same completeness as the t_1 snapshot. Y is the recorded ground-truth state against which replay will be compared.

(4) Record provenance for all changes from t_1 to t_2 : writer, timestamp, rule reference where applicable, antecedent references, and recorded LLM output where the change was cell-mediated. The recording is the substrate's normal writing discipline, not a retrospective reconstruction.

(5) Replay changes from X . Initialize a replay environment to X . Apply recorded changes in recorded order. For human-initiated changes, apply the recorded write directly. For cell-mediated changes whose rules invoked the LLM, replay the recorded LLM output rather than calling the LLM again — the recorded output is the input to the cell's deterministic transformation. For cell-mediated changes whose rules did not invoke the LLM, the rule's deterministic logic over substrate state suffices.

(6) Verify that replay produces substrate state Y' equal to Y . Equality is judged at the substrate-content layer, not at any natural-language wrapping a particular cell or LLM may have produced.

(7) Verify that intermediate states during replay match recorded intermediates. Truncating the recorded change sequence at each write produces an intermediate the replay must match. This catches reproducibility failures whose final state happens to coincide with Y by accident — non-deterministic intermediates whose effects get overwritten by later writes.

The test passes if the procedure produces equality at steps 6 and 7; otherwise it fails.

4. What the test outputs

The test produces a binary outcome with diagnostic resolution where it fails.

Pass. Replay reproduces Y' equal to Y , and intermediate states match the recorded intermediates.

Fail. Any of the following: final-state divergence (Y' differs from Y); intermediate-state divergence (a replayed intermediate differs from the recorded intermediate even when the final state matches); incomplete provenance; missing recorded LLM outputs; replay-engine failure (replay cannot execute a recorded change because recorded provenance does not specify enough of its inputs).

The diagnostic resolution — which fail mode applied — is itself recoverable from substrate content. Each fail mode points to a specific anti-pattern, named below.

5. What anti-patterns the test detects

The test detects eight canonical anti-patterns, each producing one of the failure modes in §4.

Implicit context in cells. Cell behavior depends on context that is neither in substrate state nor in the orchestration rule and is not recorded as a provenance input. Replay produces different state because the implicit context is not reproduced.

Hidden cell state. Cell-internal state persists across executions and affects subsequent writes. A cell that "remembers" prior runs in a way the substrate does not record will replay differently.

LLM-as-terminal-producer. LLM outputs flow through cells that do not transform them under deterministic rule logic — the cell writes the LLM output without rule-mediated transformation, propagating the LLM's non-determinism. Replay with the recorded output reproduces the recorded write, but the original execution carried non-determinism through to the substrate as if it were deterministic content. The diagnostic clue is failures clustering at cells whose rules are pass-through wrappers.

Non-addressable writes. A change to substrate content lacks one or more required provenance fields. Detected at step 4 — there is no input specification for the change — or at step 5 if replay is attempted anyway. Most often arises when an automated pipeline writes to the substrate without recording its own provenance, or when an LLM modifies substrate state without rules requiring an addressable origin.

Missing recorded LLM outputs. A cell-mediated change invoked the LLM, but the output was not recorded. Detected when replay cannot proceed deterministically through the affected step. Distinct from non-addressable writes: writer, timestamp, rule, and antecedents may all be recorded; what is missing is the LLM output, which is required because LLM outputs are bounded non-determinism inputs to deterministic cell logic.

Time-dependent behavior. Cells consult wall-clock time as input to substrate writes without recording the time consulted. Replay at a different wall-clock time observes a different time and produces different writes.

Environment-dependent behavior. Cells consult environmental state (file system, network, external service responses) without recording the consulted values. The replay environment does not reproduce arbitrary environmental state.

Partial provenance. A subset of provenance fields is recorded across the interval, but the subset is insufficient for replay — for example, writer and timestamp are present but antecedent references are omitted on a class of writes. More common than missing provenance, because schemas often specify required fields but cell-layer rules relax the specification under particular conditions.

The test's diagnostic value is that each fail mode points to a specific architectural anti-pattern, and the remediation is a specific architectural change rather than a workaround.

6. How the test integrates with deployment verification

The test is exercised in six deployment-verification contexts.

Initial deployment validation, before activation, on a representative interval drawn from staging. A pass demonstrates the composition holds end-to-end; a fail identifies a defect to remediate.

Post-incident verification, after any substrate-state discrepancy. A pass confirms the recorded history is reconstructible; the discrepancy is therefore explained by the recorded changes themselves. A fail identifies the anti-pattern that made the recorded history irreconstructible — often the same defect as the incident.

Architectural-change verification, after modifications to cell logic, orchestration rules, or substrate schema. A fail flags a regression in the integrative property even where component tests still pass. This is the most common context in which reproducibility regressions are introduced.

LLM-vendor-update verification, when the LLM is updated. Pre-update recorded LLM outputs are used as inputs to replay, verifying that recorded outputs continue to enable replay across the version change — the LLM-vendor independence the architecture commits to.

Composition-partner verification, when the deployment composes with another CKS substrate or an adjacent substrate. The most architecturally subtle reproducibility failures arise at composition boundaries.

Audit reconstruction. When an audit requires reconstructing a historical state — for compliance, dispute resolution, regulatory inquiry, or forensic investigation — replay from a snapshot prior to the relevant interval. A successful audit reconstruction is a successful reproducibility test on the audited interval; this is what makes audit reconstruction a procedural exercise rather than an investigative one.

7. Limits of the test

The test has narrow architectural scope. Three limits are explicit.

The test does not verify component tests in isolation. Path retraceability has its own test; the determinism contract decomposes into read determinism, cell-behavior determinism, write addressability, conflict preservation, and substrate-as-source-of-truth tests; provenance completeness and the four accountability questions have their own tests. Those tests are siblings, not subroutines. A pass on A5.16 demonstrates the composition holds; verifying components independently requires the component tests.

The test does not verify that recorded changes were correct. Reproducibility is a property of the recording-and-replay relationship, not of content correctness. A recording of a wrong decision replays as faithfully as a recording of a correct one. Correctness verification requires other instruments.

The test does not verify that the reproduced state matches external truth. The reproduced t2 state matches the recorded t2 state; this is the integrative property the test certifies. Whether either matches operators' actual decisions, a regulator's expectation, or the system's intended behavior is outside scope.

These limits are not weaknesses but the precise scope of the commitment under test.

8. Operational test

A CKS deployment satisfies the reproducibility test over a recorded interval $[t_1, t_2]$ iff:

1. Substrate states at t1 and t2 are each captured completely, at the source-of-truth content scope.
2. Every change between t1 and t2 carries provenance sufficient for replay: writer, timestamp, rule reference, antecedent references, and recorded LLM output where cell-mediated.
3. Replay from X at t1, applying recorded changes in order with recorded LLM outputs replayed at the relevant steps, produces Y' equal to Y.
4. Every intermediate state during replay matches the corresponding recorded intermediate.
5. The procedure (1)–(4) is executable from substrate content alone, without external logs, agent memory, or operator recollection.

A deployment that fails any of (1)–(5) over a representative interval does not satisfy the reproducibility commitment for that interval, regardless of component-test results.

9. Why naming the test as standalone matters, and what closes Phase A5

Reproducibility is the integrative property the source paper's foundational commitments produce when they compose. Component tests verify that each commitment holds; the integrative test verifies that the composition holds. Without it, deployments can certify each component and still fail the property the components were committed to produce. The standalone treatment gives implementers a single procedural specification, gives auditors a single citable target, and gives critics a single procedure to argue with.

A5.16 closes Phase A5. The phase's sixteen notes formalize the operational tests as standalone publications: human-governed rights individually and jointly, the substrate-cell boundary, conflict preservation, AI-as-substrate-mediator, tool-agnosticism, substrate-as-source-of-truth, the determinism contract's components, accountability and provenance completeness, and the composition cluster — composition requirements, pattern-mapping, and the integrative reproducibility test specified here. Phase A5 thereby provides a comprehensive deployment-verification toolkit.

After A5.16, Series A continues with Phase A6 (~15 notes). The programme then proceeds to Series B (~210 notes), Series C (~30 notes), and Series D/E/F (~220–260 notes), each using the same six-phase structure with its own integrative closing test.

Subsequent work that implements, extends, or argues against the CKS reproducibility commitment should use *the reproducibility test* in the sense formalized here. Work that uses the term differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Reproducibility Test as Standalone Integrative Procedure Specification: Formalizing the Operational Test That Verifies A1.07 × A1.10 Composition Through Replay-Verification in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.